Lab #1

Group number : 5

Group members : Aditya Ray, Krutyanjay Shinde

1. Prelab

1. **What flag for ls displays all files, even hidden ones?**
   ⇒ The command is `ls -al`
   - The `-a` flag stands for “all” that shows hidden files
   - The `-l` flag provides a "long" listing format, showing details like permission, owner, size, and last modified date.
2. **How do you move to the parent directory of the current one?**
   ⇒ You can move to the parent directory of your current location with the command `cd ..`
   - In file paths, a single dot (.) represents the current directory, while a double dot (..) represents the parent directory.
3. **What is the difference between the mv and cp commands?**
   ⇒ `mv` command is used to move or rename files and directories, while the `cp` command is used to copy them.
   - `mv <old_name> <new_name>`: This renames the file.
   - `mv <file_name> /move_to_directory_name/`: This moves the file to a new location. After the command, the original file is gone.
   - `cp <file_name> /move_to_directory_name/`: This creates a duplicate of the file in the new location.
4. **What does the -h flag of ls do? Hint: use man to find out.**
   ⇒ The `-h` flag prints file and folder sizes in a "human-readable" format instead of just showing the number of bytes.

   E.g., instead of 131072, we will get something like 128M.

5. **In a git repository, what command displays whether there are any local changes and what they are?**
   ⇒ The `git status` command displays whether there are any local changes in the repository and what their status is

   E.g. ., staged, unstaged, or untracked.

6. **In a git repository, what does git pull do?**
   ⇒ The `git pull` command makes your local repository same as that of the remote repository. I.e., it replaces the changed files in your local repo with whatever its in remote.

2. Checkoffs

1. **Verify that the VS Code and command-line mechanisms are working. Explain how the timing of the blinking of the LED is controlled.**
   ⇒ The timing of the LED is controlled by a timer with a specified period and offset.
   `timer t(offset, period)`
   This timer then triggers a reaction that flips the state of the LED at the specified period and prints the state.
2. **Show your modified LF program. Explain how this use of timers is different from the sleep function used in the C code blink.c.**
   ⇒ In C, `sleep()` halts the program, making the CPU idle. Lingua Franca timers, however, schedule events in "logical time." This means they don't stop the program; instead, they trigger actions at a specific future logical time, allowing the system to handle other tasks concurrently.

   ⇒ToolsLEDSolution.lf

```
target C {
  platform: {
    name: "rp2040",
    board: "pololu_3pi_2040_robot"
  },
  single-threaded: true
}

preamble {=
  #include <stdio.h>
  #include <pico/stdlib.h>
  #include <hardware/gpio.h>
=}

main reactor {
  timer ton(0, 200ms)
  timer toff(100ms, 200ms)
```

```
  reaction(startup) {=
    gpio_init(PICO_DEFAULT_LED_PIN);
    gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
  =}

  reaction led_on(ton) {=
    printf("LED State: %b\n", true);
    gpio_put(PICO_DEFAULT_LED_PIN, true);
  =}

  reaction led_off(toff) {=
    printf("LED State: %b\n", false);
    gpio_put(PICO_DEFAULT_LED_PIN, false);
  =}
}
```

⇒**ToolsPrintfSolution.lf**

```
target C {
  platform: {
    name: "rp2040",
    board: "pololu_3pi_2040_robot"
  },
  single-threaded: true
}

preamble {=
  #include <stdio.h>
  #include <pico/stdlib.h>
  #include <hardware/gpio.h>
```

```
=}

main reactor {
  timer t(0, 500 ms)
  state led_on: bool = false

  reaction(startup) {=
    // Initialize stdio for printf over USB.
    stdio_init_all();

    // Initialize the GPIO pin for the LED.
    gpio_init(PICO_DEFAULT_LED_PIN);
    gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
  =}

  reaction(t) {=
    // Toggle the state for the next blink.
    self->led_on = !self->led_on;

    if (self->led_on) {
      printf("ON\n");
      gpio_put(PICO_DEFAULT_LED_PIN, 1);
    } else {
      printf("OFF\n");
      gpio_put(PICO_DEFAULT_LED_PIN, 0);
    }
  =}
}
```

3. **Show ON-OFF output.**
   ⇒ Shown in in-person check-off.
4. **Show and explain how your program works.**
   ⇒ (Shown in in-person check-off)

   ⇒ ToolsLEDSolution.lf

   Component 1: The `LED` Reactor

   The `LED.lf` file acts as a simple, reusable hardware controller for the physical LED.

   - It has a boolean input named `set` to receive on/off commands.
   - A `startup` reaction initializes the GPIO pin connected to the LED .
   - A second reaction triggers whenever a new boolean value arrives on the `set` input. It then uses `gpio_put` to turn the physical LED on or off to match that value.

Component 2: The led_on Driver

The `led_on` reactor contains the timing logic that determines *when* the LED should blink.

- It uses a timer `t` that triggers every 250ms.
- It has a boolean output named `set`.
- Every time the timer triggers, its reaction sends out the logical opposite of its previous output value (`!set->value`) . This creates a toggling `true`/`false` signal every 250ms.

The `main` reactor connects these two components to create the final blinking behavior.

1. It creates an instance of the `LED` controller named `led`.
2. It creates an instance of the timing logic named `led_driver`.
3. It connects the output of the driver to the input of the controller with the line `led_driver.set -> led.set`.

⇒ ToolsPrintfSolution.lf

Component 1: The `main` Reactor

The `main reactor` is the central component of the program. It defines the state and the events that will drive the application's logic.

- `timer t(0, 500 ms)`: This line creates a timer, which is an event source. It's configured to trigger its first event immediately at time `0` and then trigger repeatedly every 500 milliseconds.

- `state led_on: bool = false`: This line declares a private variable for the reactor that acts as its memory. It's a boolean named `led_on`, and its job is to keep track of whether the LED should be on or off. It is initialized to `false`.

Component 2: The Reactions

The reactions contain the code that executes when the events defined in the reactor occur.

- `reaction(startup)`: This reaction runs only once when the program begins. Its purpose is to perform one-time setup. It calls `stdio_init_all()` to enable the USB serial port so `printf` will work, and it calls `gpio_init()` and `gpio_set_dir()` to configure the physical LED pin as an output.
- `reaction(t)`: This reaction is triggered by the timer `t`, meaning its code runs every 500 milliseconds.
    1. It first flips the value of the reactor's `led_on` state variable.
    2. It then checks the new state: if `led_on` is now `true`, it prints "ON" and turns the physical LED on; otherwise, it prints "OFF" and turns the LED off.

3. Postlab

1. **What format specifier(s) for printf allows the printing of floats (there may be several)?**
   ⇒ The primary format specifier for floats is `%f`. You can also control the number of decimal places shown, for instance, `%.2f` displays two decimal places, and `%.4f` displays four.

   Other specifiers for floating-point numbers include:
   - `%e`: For scientific notation (e.g., 1.2345e+02).
   - `%g`: Automatically uses the more compact form between `%f` and `%e`

2. **When might printf statements be the best tool for debugging?**
   ⇒ `printf` statements are often the best debugging tool when we have access to a standard output stream like a serial terminal.
   - It's a simple way to verify program flow and inspect the value of variables at specific points in our code.

3. **What other tools might be useful for debugging embedded software (note that using an interactive debugger like gdb with the robot or Pico board is possible but requires extra hardware)?**
   ⇒ Besides `printf`, several other tools are useful for debugging embedded systems:
   - **GPIO Pins**: We can program GPIO pins to turn on or off when certain things happen in our code, by connecting an oscilloscope.
   - **Cross-Debugger**:We can run a "GDB server" program on the Pico, which communicates with the full GDB cross-debugger running on our main development computer. The two are connected via a USB cable. This setup lets us control the program on the Pico our your PC.
4. **What does the volatile keyword mean in C and when might you use it?**
   ⇒ `volatile` keyword tells the compiler **not to optimize a variable**.
   - **The Problem**: A compiler will often optimize code by storing a variable's value in a fast CPU register, assuming it won't change unexpectedly. This is a problem if the variable is tied to hardware (like a status register) that can change its value at any moment.
   - **The Solution**: `volatile` forces the compiler to re-read the variable from its main memory address every single time it's used. This guarantees the program always has the most up-to-date value, which is critical when working with hardware or in Interrupt Service Routines (ISRs)
5. **What were your takeaways from the lab? What did you learn during the lab? Did any results in the lab surprise you?**
   ⇒ During this lab, we learned the complete workflow for setting up a cross-compilation development environment for a bare-metal embedded target (raspberry pi pico). This involved configuring Windows Subsystem for Linux (WSL) and using the Nix package manager to create a reproducible shell with all the necessary tools, such as the ARM cross-compiler and `picotool`. A key part of the setup was learning to manage hardware permissions by creating `udev` rules, which are necessary to allow user-level access to the device's USB interface without requiring `sudo`.

   ⇒ On the programming side, we learned how Lingua Franca provides a structured, event-driven approach for embedded software. We were introduced to its major components, including **reactors**, which are composable blocks of code that communicate through inputs and outputs, and **reactions**, which are the event-triggered functions that contain the application logic. We also learned how the execution of these reactions is precisely scheduled based on **logical time** using **timers**, which is fundamentally different from using blocking functions like `sleep()` in traditional C programming.